

Masks, Per-Location Prediction, and Segmentation Losses

Mathematics of Deep Learning — Chapter 17

Yin Yang

Foundation Books Project

Roadmap

From Image Labels to Pixel Labels

Mask Tensors and Shapes

Pixelwise Cross-Entropy

Imbalance and Reweighting

Overlap Losses: Dice and IoU

Tversky and Cost Asymmetry

Resolution and Upsampling

Encoder-Decoder and Skips

Boundary-Aware Objectives

Instance Segmentation and Matching

Panoptic Segmentation

Query-Based and Promptable

Evaluation and Calibration

Summary

From Image Labels to Pixel Labels

Why Masks, Not Just Boxes?

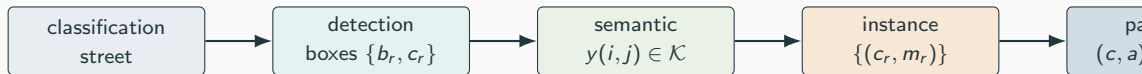
A driving system needs more than “there is a car somewhere.”

- Classification: one label per image.
- Detection: a few boxes around objects.
- Semantic segmentation: *one class per pixel*.
- Instance segmentation: *one mask per object*.
- Panoptic segmentation: *one full scene partition*.

Useful answers are spatial: drivable road area, sidewalk edge, the region occupied by each pedestrian.

Segmentation predicts a spatial field — one decision for every relevant image location.

Four Outputs for One Image



Same street scene answers different practical questions:

- What kind of image is this?
- Where are the objects roughly?
- Which pixels are road / sky / car?
- Which pixels belong to *this* car vs. *that* car?
- One coherent partition of the whole scene.

The practical question determines the target.

Mask Tensors and Shapes

Label Maps and Ignore Pixels

Pixel-location set: $\Omega = \{1, \dots, H\} \times \{1, \dots, W\}$. Semantic label map:

$$y : \Omega \rightarrow \{0, 1, \dots, K - 1\} \cup \{\text{ign}\}.$$

Valid-pixel set:

$$\Omega_{\text{valid}} = \{(i, j) \in \Omega : y(i, j) \neq \text{ign}\}.$$

- `ign` marks unlabeled or ambiguous pixels.
- Ignored is *not* the same as background — background is an ordinary class.
- The loss averages only over valid pixels.

Most segmentation bugs are shape and bookkeeping bugs — `ign` is bookkeeping made explicit.

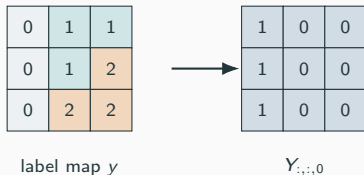
Label Map to One-Hot Mask Tensor

0	1	1
0	1	2
0	2	2

label map y

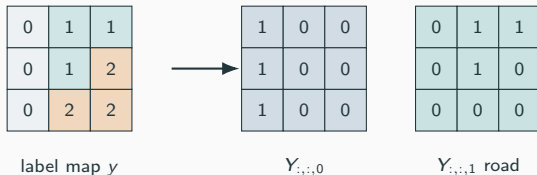
(1) Stored label map: grid of class IDs.

Label Map to One-Hot Mask Tensor



- (1) Stored label map: grid of class IDs.
- (2)–(4) One-hot expansion: one binary channel per class.

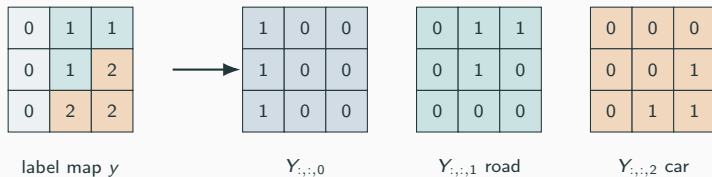
Label Map to One-Hot Mask Tensor



(1) Stored label map: grid of class IDs.

(2)–(4) One-hot expansion: one binary channel per class.

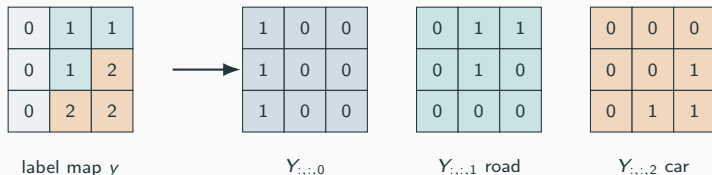
Label Map to One-Hot Mask Tensor



(1) Stored label map: grid of class IDs.

(2)–(4) One-hot expansion: one binary channel per class.

Label Map to One-Hot Mask Tensor



Batch tensors:

$$X \in [0, 1]^{B \times H \times W \times C}$$

$$Z \in \mathbb{R}^{B \times H \times W \times K}$$

logits, labels, masks must align

- (1) Stored label map: grid of class IDs.
- (2)–(4) One-hot expansion: one binary channel per class.
- (5) A batched logit tensor stacks all of these per example.

Spatial axes must match. A 1-pixel offset becomes a bookkeeping error punished as a recognition error.

Pixelwise Cross-Entropy

Pixelwise Logits, Softmax, Cross-Entropy

At each valid pixel, logits $z_{i,j,:} \in \mathbb{R}^K$ and probabilities

$$p_{i,j,k} = \frac{\exp z_{i,j,k}}{\sum_{\ell} \exp z_{i,j,\ell}} = \text{softmax}(z_{i,j,:})_k.$$

Average pixelwise cross-entropy over valid pixels:

$$\mathcal{L}_{\text{CE}} = -\frac{1}{|\Omega_{\text{valid}}|} \sum_{(i,j) \in \Omega_{\text{valid}}} \log p_{i,j,y(i,j)}.$$

Logit gradient at one valid pixel:

$$\frac{\partial \ell_{i,j}}{\partial z_{i,j,k}} = p_{i,j,k} - \mathbf{1}\{k = y(i,j)\}.$$

Pixelwise CE is image-level classification copied across a grid — one local correction per valid pixel.

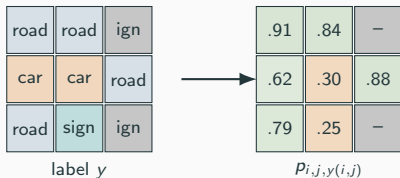
Pixelwise CE on a Tiny Grid, Step by Step

road	road	ign
car	car	road
road	sign	ign

label y

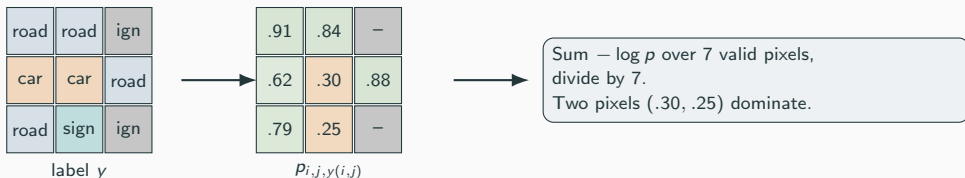
(1) Label map has ignored pixels (gray).

Pixelwise CE on a Tiny Grid, Step by Step



- (1) Label map has ignored pixels (gray).
- (2) Read correct-class probability *only* at valid pixels.

Pixelwise CE on a Tiny Grid, Step by Step



- (1) Label map has ignored pixels (gray).
- (2) Read correct-class probability *only* at valid pixels.
- (3) Average $-\log p$ over the 7 valid pixels.

Low correct-class probabilities at valid pixels contribute large local terms; ignored pixels contribute zero.

Imbalance and Reweighting

Class Imbalance and Weighted CE

Empirical class frequency:

$$\hat{\pi}_k = \frac{|\{(n, i, j) : y_n(i, j) = k\}|}{|\{(n, i, j) : y_n(i, j) \neq \text{ign}\}|}.$$

A pedestrian or traffic sign may be rare in pixels and yet critical.

Weighted pixelwise CE:

$$\mathcal{L}_{\text{WCE}} = -\frac{1}{|\Omega_{\text{valid}}|} \sum_{(i,j) \in \Omega_{\text{valid}}} w_{y(i,j)} \log p_{i,j,y(i,j)}.$$

Pixel-accuracy trap: 95% accuracy from predicting background everywhere can hide 0% foreground recall.

Weights make rare-but-important mistakes count more — not a free lunch, just a different optimization problem.

Focal Reweighting Damps Easy Pixels

Focal factor with $p_t = p_{i,j,y(i,j)}$:

$$\ell_{\text{focal}} = -(1 - p_t)^\gamma \log p_t.$$

With $\gamma = 2$:

p_t	$-\log p_t$	$(1 - p_t)^2$	focal $\cdot$ CE
0.90	0.105	0.010	0.001
0.50	0.693	0.250	0.173
0.10	2.303	0.810	1.865

Easy pixels (p_t near 1) almost vanish from the loss; hard pixels dominate.

Focal: shift gradient budget from easy backgrounds toward uncertain foreground.

Overlap Losses: Dice and IoU

Soft Counts, Dice, and IoU

For binary mask, pixel-indexed $y_u \in \{0, 1\}$, $s_u \in [0, 1]$:

$$\text{TP} = \sum_u s_u y_u, \quad \text{FP} = \sum_u s_u (1 - y_u), \quad \text{FN} = \sum_u (1 - s_u) y_u.$$

Dice (soft, with smoothing ϵ)

$$\text{Dice} = \frac{2 \text{TP} + \epsilon}{2 \text{TP} + \text{FP} + \text{FN} + \epsilon}.$$

Jaccard / IoU

$$\text{IoU} = \frac{\text{TP} + \epsilon}{\text{TP} + \text{FP} + \text{FN} + \epsilon}.$$

Identity: $\text{Dice} = \frac{2 \text{IoU}}{1 + \text{IoU}}$ — same ranking, different gradients.

Overlap losses score regions; their denominators couple all pixels of one mask.

Building Up an Overlap Score

TN	FN	FN	TN
FP	TP	TP	TN
FP	TP	TP	FP
TN	TN	TP	TN

overlap categories

(1) Partition pixels into TP/FP/FN/TN.

Building Up an Overlap Score

TN	FN	FN	TN
FP	TP	TP	TN
FP	TP	TP	FP
TN	TN	TP	TN

overlap categories

Hard counts:

$TP = 5$, $FP = 3$, $FN = 2$

- (1) Partition pixels into TP/FP/FN/TN.
- (2) Count the categories.

Building Up an Overlap Score

TN	FN	FN	TN
FP	TP	TP	TN
FP	TP	TP	FP
TN	TN	TP	TN

overlap categories

Hard counts:

TP = 5, FP = 3, FN = 2

$$\text{Dice} = \frac{10}{10 + 3 + 2} = 0.80$$

$$\text{IoU} = \frac{5}{5 + 3 + 2} = 0.5$$

- (1) Partition pixels into TP/FP/FN/TN.
- (2) Count the categories.
- (3) Plug into Dice and IoU formulas.

Building Up an Overlap Score

TN	FN	FN	TN
FP	TP	TP	TN
FP	TP	TP	FP
TN	TN	TP	TN

overlap categories

Hard counts:

TP = 5, FP = 3, FN = 2

$$\text{Dice} = \frac{10}{10 + 3 + 2} = 0.80$$

$$\text{IoU} = \frac{5}{5 + 3 + 2} = 0.5$$

Loss: $1 - \text{Dice}$ or $1 - \text{IoU}$. Changing one pixel changes a ratio whose denominator depends on the *whole* mask — pixels are coupled through the score.

- (1) Partition pixels into TP/FP/FN/TN.
- (2) Count the categories.
- (3) Plug into Dice and IoU formulas.
- (4) Gradients are global, not pixel-local.

Generalized Dice for Rare Classes

Multi-class soft predictions $s_{u,c}$, one-hot labels $y_{u,c}$, class weights w_c :

$$\text{GDice}(s, y) = \frac{2 \sum_c w_c \sum_u s_{u,c} y_{u,c}}{\sum_c w_c \sum_u (s_{u,c} + y_{u,c})}.$$

Common choice: $w_c = 1/(\sum_u y_{u,c})^2$ (inverse squared volume).

Example: class 1 has 10^3 pixels, class 2 has 10 pixels:

$$w_1 = 10^{-6}, \quad w_2 = 10^{-2}.$$

An error on the rare class is no longer washed out by the common one.

Generalized Dice rebalances the overlap budget across classes — a region-level analogue of weighted CE.

Tversky and Cost Asymmetry

Precision, Recall, and the Tversky Index

$$\text{precision} = \frac{TP}{TP + FP}, \quad \text{recall} = \frac{TP}{TP + FN}.$$

Tversky index, $\alpha, \beta \geq 0$

$$T_{\alpha, \beta} = \frac{TP + \epsilon}{TP + \alpha FP + \beta FN + \epsilon}.$$

Special cases (no smoothing):

$$T_{1/2, 1/2} = \text{Dice}, \quad T_{1, 1} = \text{IoU}.$$

Raising β penalizes missed foreground (FN) more. Raising α penalizes leakage (FP) more.

Different applications need different (α, β) : a screening model is not a photo-editor.

Cost Asymmetry, Step by Step

TN	TN	FP
FP	TP	TP
TN	TP	FP

leak case

α large

(1) Leakage failure: extra foreground.

Cost Asymmetry, Step by Step

TN	TN	FP
FP	TP	TP
TN	TP	FP

leak case
 α large

TN	FN	TN
TN	TP	TP
TN	FN	TN

miss case
 β large

- (1) Leakage failure: extra foreground.
- (2) Miss failure: foreground gone.

Cost Asymmetry, Step by Step

TN	TN	FP
FP	TP	TP
TN	TP	FP

leak case
 α large

TN	FN	TN
TN	TP	TP
TN	FN	TN

miss case
 β large

Same TP, but different denominators.

With $(\alpha, \beta) = (0.3, 0.7)$:

leak: $18/(18 + 0.3 \cdot 6 + 0.7 \cdot 3) \approx 0.822$

miss: $18/(18 + 0.3 \cdot 3 + 0.7 \cdot 6) \approx 0.779$

- (1) Leakage failure: extra foreground.
- (2) Miss failure: foreground gone.
- (3) Tversky scores them differently via α, β .

Cost Asymmetry, Step by Step

TN	TN	FP
FP	TP	TP
TN	TP	FP

leak case
 α large

TN	FN	TN
TN	TP	TP
TN	FN	TN

miss case
 β large

Same TP, but different denominators.

With $(\alpha, \beta) = (0.3, 0.7)$:

leak: $18 / (18 + 0.3 \cdot 6 + 0.7 \cdot 3) \approx 0.822$

miss: $18 / (18 + 0.3 \cdot 3 + 0.7 \cdot 6) \approx 0.779$

Missing foreground penalized more when $\beta > \alpha$ — the loss now matches the cost.

- (1) Leakage failure: extra foreground.
- (2) Miss failure: foreground gone.
- (3) Tversky scores them differently via α, β .
- (4) Match the loss to the asymmetric deployment cost.

Resolution and Upsampling

Output Stride and Alignment

Output stride s : ratio of input pixel spacing to output cell spacing.

$$H \times W \longrightarrow (H/s) \times (W/s).$$

Input	Stride	Coarse H'	Coarse W'
128×128	8	16	16
128×128	16	8	8
256×256	8	32	32
256×256	32	8	8

Upsampling restores resolution. Label maps resize with *nearest-neighbor* — class IDs are not continuous.

Strided features are fine for context; the loss still needs pixel-aligned predictions.

Encoder-Decoder and Skips

Encoder-Decoder with Skip Connections



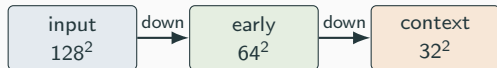
(1) Input grid carries colour and pixel locations.

Encoder-Decoder with Skip Connections



- (1) Input grid carries colour and pixel locations.
- (2) Encoder downsamples for context (*what*).

Encoder-Decoder with Skip Connections



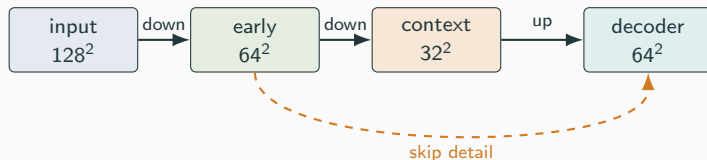
- (1) Input grid carries colour and pixel locations.
- (2) Encoder downsamples for context (*what*).
- (3) Bottleneck: largest receptive field, smallest grid.

Encoder-Decoder with Skip Connections



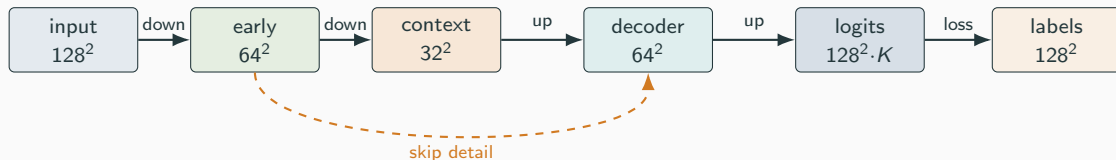
- (1) Input grid carries colour and pixel locations.
- (2) Encoder downsamples for context (*what*).
- (3) Bottleneck: largest receptive field, smallest grid.
- (4) Decoder upsamples toward pixel resolution.

Encoder-Decoder with Skip Connections



- (1) Input grid carries colour and pixel locations.
- (2) Encoder downsamples for context (*what*).
- (3) Bottleneck: largest receptive field, smallest grid.
- (4) Decoder upsamples toward pixel resolution.
- (5) Skip connections inject high-resolution detail (*where*).

Encoder-Decoder with Skip Connections



- (1) Input grid carries colour and pixel locations.
- (2) Encoder downsamples for context (*what*).
- (3) Bottleneck: largest receptive field, smallest grid.
- (4) Decoder upsamples toward pixel resolution.
- (5) Skip connections inject high-resolution detail (*where*).
- (6) Logits are aligned with the label map for the loss.

Encoder answers what; skips and decoder answer where.

Boundary-Aware Objectives

Boundary and Shape: Beyond Average Overlap

Two masks can share Dice yet differ in boundary quality.

Signed distance transform $\phi_y(u)$: negative inside target, positive outside. Distance-based boundary loss:

$$\mathcal{L}_{\text{boundary}}(s, y) = \sum_{u \in \Omega} s_u \phi_y(u).$$

Hausdorff distance for boundaries A, B :

$$d_H(A, B) = \max \left\{ \max_{a \in A} \min_{b \in B} d(a, b), \max_{b \in B} \min_{a \in A} d(a, b) \right\}.$$

Worst boundary mismatch — sensitive to outliers.

Same area can hide topology errors and far-away outliers — add boundary diagnostics when shape matters.

Instance Segmentation and Matching

Mask-Level Matching for Instances

Predictions $\mathcal{P} = \{(\hat{c}_r, \hat{m}_r, \hat{s}_r)\}_{r=1}^R$, targets $\mathcal{G} = \{(c_q, m_q)\}_{q=1}^Q$.

IoU-threshold matching: $\hat{m}_r$ matches m_q only if classes agree and $\text{IoU}(\hat{m}_r, m_q) \geq \tau$.

Bipartite assignment minimizes a cost $C_{r,q}$:

$$\min_{\sigma} \sum_{q=1}^Q C_{\sigma(q),q},$$

with C combining class CE, mask BCE, and Dice loss.

Under one-to-one matching: $\# \text{ TP} \leq Q$ — a duplicate mask cannot steal a second credit for the same pedestrian.

Counting objects forces a matching layer between predictions and targets.

Matching a Tiny IoU Matrix, Step by Step

		car A	car B
pred ↓ / tgt	pred 1	0.80	0.10
	pred 2	0.55	0.20
	pred 3	0.05	0.70

(1) Build mask-vs-mask IoU matrix.

Matching a Tiny IoU Matrix, Step by Step

	car A	car B
pred ↓ / tgt		
pred 1	0.80	0.10
pred 2	0.55	0.20
pred 3	0.05	0.70

Threshold $\tau = 0.5$.

Best match for car A: pred 1.

Best match for car B: pred 3.

- (1) Build mask-vs-mask IoU matrix.
- (2) Pick best eligible match for each target.

Matching a Tiny IoU Matrix, Step by Step

	car A	car B
pred ↓ / tgt		
pred 1	0.80	0.10
pred 2	0.55	0.20
pred 3	0.05	0.70

Threshold $\tau = 0.5$.
Best match for car A: pred 1.
Best match for car B: pred 3.

Pred 2 also overlaps car A above τ ,
but car A is taken: pred 2 is a
duplicate $\Rightarrow$ false positive.

- (1) Build mask-vs-mask IoU matrix.
- (2) Pick best eligible match for each target.
- (3) Extra overlapping predictions become FP duplicates.

Matching a Tiny IoU Matrix, Step by Step

	car A	car B
pred ↓ / tgt		
pred 1	0.80	0.10
pred 2	0.55	0.20
pred 3	0.05	0.70

Threshold $\tau = 0.5$.
Best match for car A: pred 1.
Best match for car B: pred 3.

Pred 2 also overlaps car A above τ ,
but car A is taken: pred 2 is a
duplicate $\Rightarrow$ false positive.

Bipartite training cost is separate from
evaluation IoU. Both are mask-level obj
not just per-pixel objects.

- (1) Build mask-vs-mask IoU matrix.
- (2) Pick best eligible match for each target.
- (3) Extra overlapping predictions become FP duplicates.
- (4) Training cost and evaluation IoU need not coincide.

Panoptic Segmentation

Panoptic: One Partition for the Whole Scene

- **Stuff** classes: amorphous regions (road, sky, water).
- **Thing** classes: countable instances (car, person).
- Panoptic label per valid pixel: (c, a) — class plus instance ID.
- Every valid pixel is in *exactly one* segment.

Panoptic Quality $PQ = SQ \cdot RQ$

$$SQ = \frac{1}{|TP|} \sum_{(p,g) \in TP} \text{IoU}(p, g), \quad RQ = \frac{|TP|}{|TP| + \frac{1}{2}|FP| + \frac{1}{2}|FN|}.$$

Example: $TP = \{0.80, 0.60\}$, $|FP| = |FN| = 1 \Rightarrow SQ = 0.70$, $RQ = 2/3$, $PQ \approx 0.467$.

SQ asks “how good are matched masks”; RQ asks “did you find the right segments”.

Query-Based and Promptable

Query-Based and Promptable Mask Prediction

A modern segmentation head predicts *a set* of masks:

$$\{(\hat{p}_r, \hat{m}_r)\}_{r=1}^N, \quad \hat{m}_r : \Omega \rightarrow [0, 1].$$

Each query is matched bipartite to a target. A typical query cost:

$$C_{r,q} = -\log \hat{p}_r(c_q) + \lambda_m \mathcal{L}_{\text{BCE}}(\hat{m}_r, m_q) + \lambda_d (1 - \text{Dice}(\hat{m}_r, m_q)).$$

Unmatched queries get a no-object target.

Promptable: mask conditioned on a point, box, or rough mask.

Prompts change the interface; the math contract is unchanged — overlap, boundaries, matching, calibration.

Evaluation and Calibration

Mean IoU, Thresholds, and Slice Reporting

Mean IoU averages classwise IoU:

$$\text{mIoU} = \frac{1}{K} \sum_{k=0}^{K-1} \text{IoU}_k.$$

Threshold t on foreground probabilities: $\hat{m}_u(t) = \mathbf{1}\{s_u \geq t\}$.

Example: $s = (0.90, 0.70, 0.40, 0.20)$, $y = (1, 1, 0, 1)$.

- $t = 0.50 \Rightarrow \text{TP} = 2, \text{FP} = 0, \text{FN} = 1.$
- $t = 0.30 \Rightarrow \text{TP} = 2, \text{FP} = 1, \text{FN} = 1.$

Lowering t added a false positive without recovering the missed pixel.

One average hides rare-class, boundary, and threshold issues — report by slice.

Calibration and Failure-Mode Checklist

Reliability bins: compare mean confidence vs. empirical accuracy. Expected Calibration Error:

$$\text{ECE} = \sum_b \frac{|B_b|}{N} |\text{acc}(B_b) - \text{conf}(B_b)|.$$

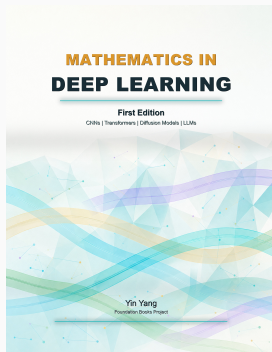
Failure mode	What to inspect
Rare class missed	Classwise IoU; recall by object size
Foreground leakage	Precision; threshold sweep
Merged instances	Matching; connected components
Bad boundaries	Boundary F-score; contour distance
Overconfident masks	Reliability bins; ECE
Dataset shift	Slice by lighting, scanner, scene type

Each diagnostic names one mathematical failure mode — not just a single average.

Summary

- **Dense prediction:** one decision per valid pixel; ignore labels are bookkeeping, not background.
- **Pixelwise CE** repeats classification across the grid; easy backgrounds can dominate.
- **Overlap losses (Dice, IoU)** score regions; gradients couple every pixel through the denominator.
- **Tversky** parameterizes false-positive vs. false-negative cost; Dice and IoU are special cases.
- **Encoder-decoder + skips:** encoder for *what*, decoder and skips for *where*.
- **Boundary losses** address shape and topology failures that overlap averages can hide.
- **Instance / panoptic / queries:** masks need matching, and the metric (PQ, mAP) must follow.
- **Evaluation:** report classwise, by slice, with thresholds and calibration — not one number.

Next: video, time, 3D, and dense prediction beyond a single image.



Mathematics in Deep Learning

Chapter overview slides for the book by Yin Yang.

Book page	foundation-books.github.io/mathematics-in-deep-learning
Code	github.com/foundation-books/mathematics-in-deep-learning-code
Paperback	amazon.com/dp/BOH1ZZHDT2
Hardcover	amazon.com/dp/BOH1ZL11MJ
Kindle	amazon.com/dp/B0GX32M8SP
License	CC BY-NC-ND 4.0

These free overview slides may be shared non-commercially with attribution and without modifications. Full explanations, exercises, and companion-code context are in the book and linked repository.